



Lab 3

TCP Sockets between WSL and Windows

Note: Mac users can do WSL experiments on their own machines and work with other classmates for Windows related activities or If you have any VM installed try the same between Mac and the VM

Mouli Sankaran

Lab 3: Focus

- WSL Explained
- Server and Client Interactions
 - Using TCP Connection between WSL and Windows
 - Connection Establishment Explained
- Analyze the TCP Segments: Wireshark
- Experiments to be done in the Lab
 - Exp 1 and Exp 2

Python IDE used on Windows: Thonny

Course page where the course materials will be posted
as the course progresses:

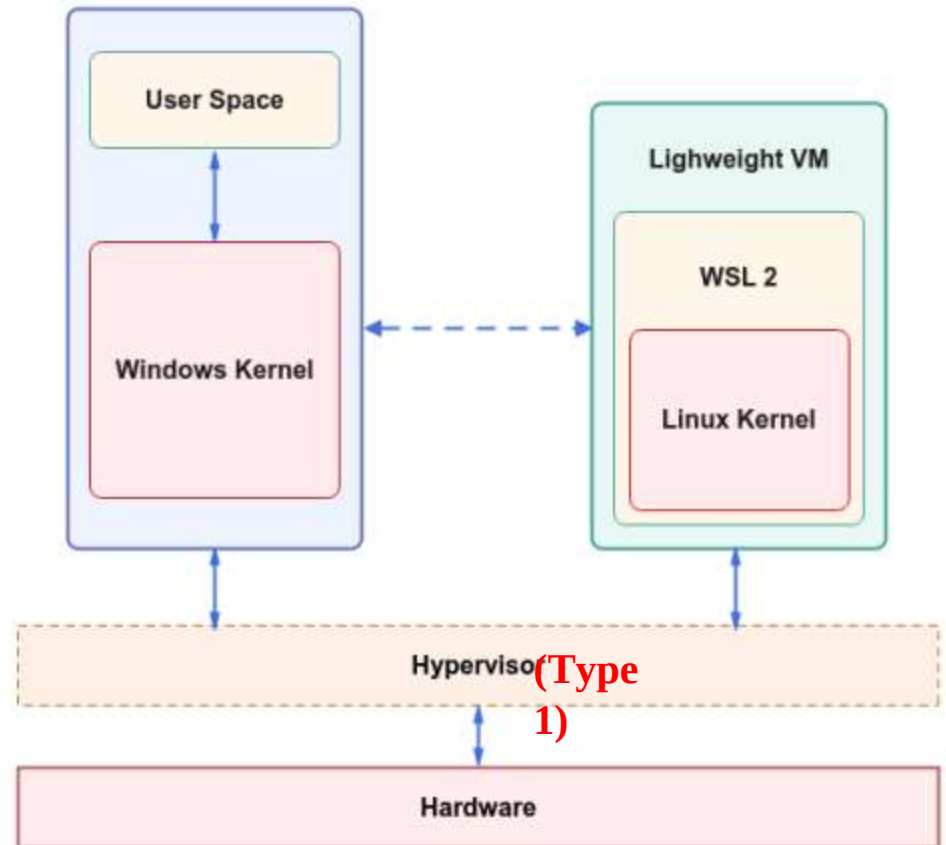


WSL Explained

WSL: Explained

Ref: [Intro to WSL 2](#)

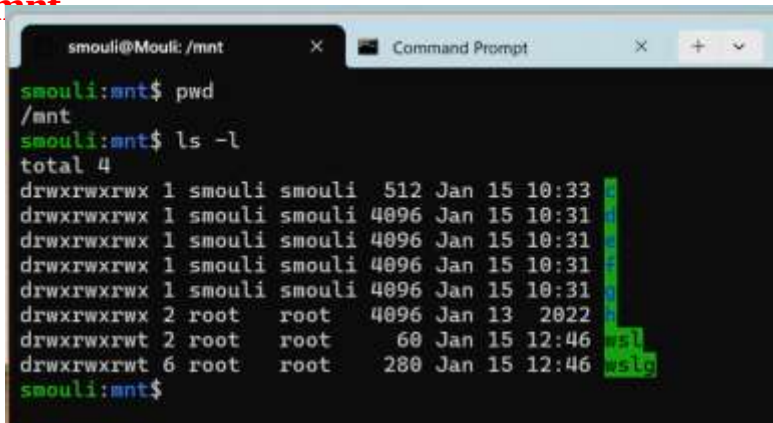
- The Windows Subsystem for Linux (WSL) allows users to run Linux applications and command-line tools on Windows.
- WSL works by translating Linux system calls into Windows system calls.
- WSL allows Linux binaries to run on Windows without the need for a virtual machine or dual booting.
- WSL manages the Linux file system, network, and process execution.
- WSL creates a Virtual Hard Disk (VHD) to store files for each Linux distribution. In our case Ubuntu.



WSL: Explained

- The Lightweight Utility VM has been optimized to load the Linux kernel into the VM's address space without going through any Boot Loader process, thereby achieving fast startup time
- WSL supports only x64 and Arm CPUs.
- Using your mounted drives, you can edit code in, for example, `C:\dev\myproj\` using Visual Studio or VS Code, and build/test that code in Linux by accessing the same files via `/mnt/c/dev/myproj`

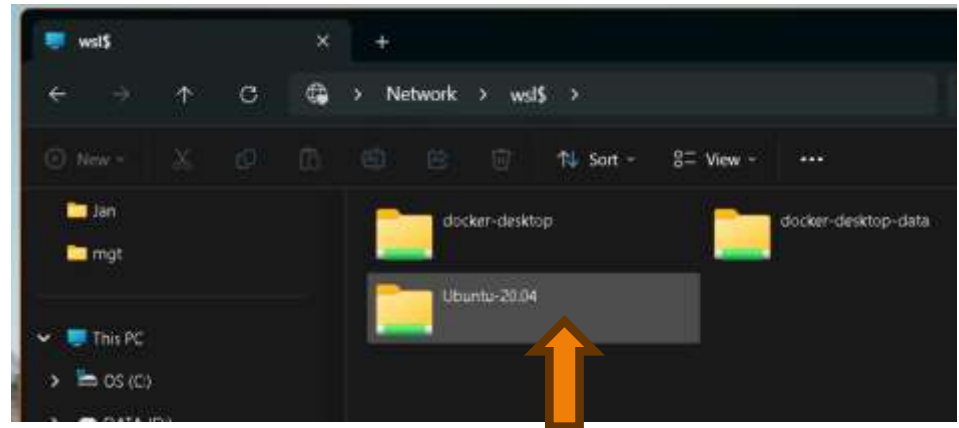
Accessing Windows filesystem from WSL:
`/mnt`



```
smouli@Mouli: /mnt
smouli:~$ pwd
/mnt
smouli:~$ ls -l
total 4
drwxrwxrwx 1 smouli smouli 512 Jan 15 10:33 mnt
drwxrwxrwx 1 smouli smouli 4096 Jan 15 10:31 mnt1
drwxrwxrwx 1 smouli smouli 4096 Jan 15 10:31 mnt2
drwxrwxrwx 1 smouli smouli 4096 Jan 15 10:31 mnt3
drwxrwxrwx 1 smouli smouli 4096 Jan 15 10:31 mnt4
drwxrwxrwx 2 root root 4096 Jan 13 2022 mnt5
drwxrwxrwt 2 root root 60 Jan 15 12:46 mnt6
drwxrwxrwt 6 root root 280 Jan 15 12:46 mnt7
smouli:~$
```

Accessing Linux filesystem (WSL) from Windows: `\\wsl$`

Enter `\\wsl$` in the File Explorer address bar to view the filesystem of Linux distribution on WSL
[Ref link.](#)



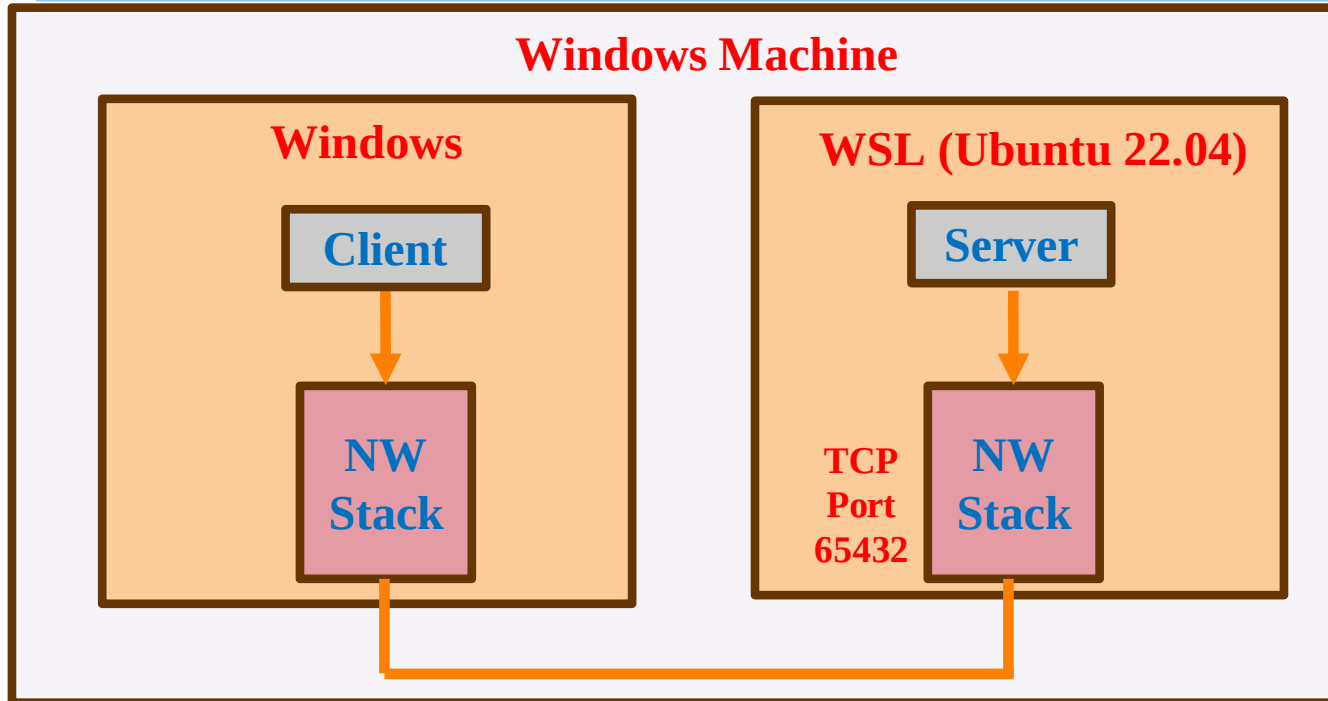


Server and Client Interactions

Reference: Ref1: TCP/IP Illustrated-Volume 1:
Chapter 13: TCP Connection Establishment and Termination

Exp: System Diagram

(sample Python code shared)



On WSL

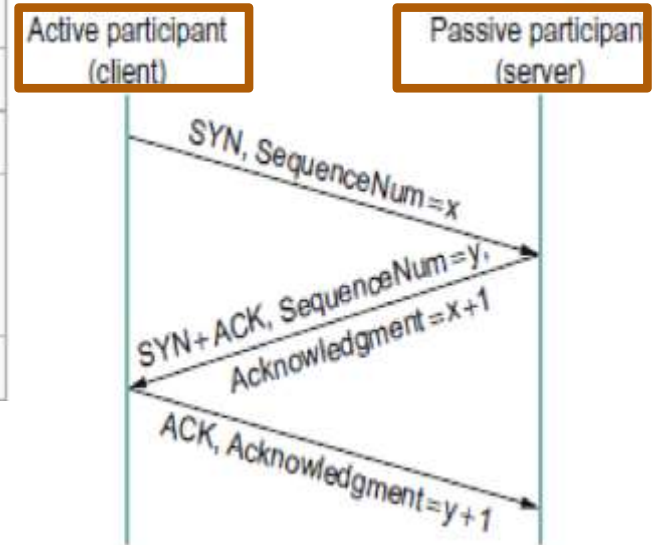


- **Exp 0:** Run the below python programs
 - **On WSL:** `Lab1_exp0_server_WSL.py` - this should be run first
 - **On Windows:** `Lab1_exp0_client_Win.py` - this should be run next

```
smouli@Mouli: ~/NetSec/Lab  ×  Command Pro
smouli:Lab1$ lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:   Ubuntu 22.04.5 LTS
Release:      22.04
Codename:     jammy
smouli:Lab1$ |
```

Connection Establishment: Explained

Source port										Destination port											
Sequence number																					
Acknowledgment number (if ACK set)																					
Data offset HDR Len	Reserved 0 0 0	N S	C	E	U	A	P	R	S	F	Window Size										
			W	C	R	C	S	S	S	S										S	I
			R	E	G	K	H	T	N	N											
Checksum										Urgent pointer (if URG set)											



● A connection typically goes through **three phases**:

1. Setup,
2. Data transfer (called *established*), and
3. Teardown (closing).

SYN: Synch or Synchronization **FIN**: Final segment of a TCP connection.

IP address: WSL (file name: Lab1_WSL.py)

- IP address of the Server running on WSL: **HOST = 172.24.215.49**

```
smouli@Mouli: ~/NetSec/Lab  X  Command Prompt  X  +  v  -  □
smouli:Lab1$ ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST>  mtu 1500
    inet 172.24.215.49  netmask 255.255.240.0  broadcast 172.24.223.255
    inet6 fe80::215:5dff:fe8a:55c7  prefixlen 64  scopeid 0x20<link>
    ether 00:15:5d:fa:55:c7  txqueuelen 1000  (Ethernet)
    RX packets 7937  bytes 810731 (810.7 KB)
    RX errors 0  dropped 0  overruns 0  frame 0
    TX packets 35  bytes 2346 (2.3 KB)
    TX errors 0  dropped 0  overruns 0  carrier 0  collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING>  mtu 65536
    inet 127.0.0.1  netmask 255.0.0.0
    inet6 ::1  prefixlen 128  scopeid 0x10<host>
    loop txqueuelen 1000  (Local Loopback)
    RX packets 4  bytes 532 (532.0 B)
    RX errors 0  dropped 0  overruns 0  frame 0
    TX packets 4  bytes 532 (532.0 B)
    TX errors 0  dropped 0  overruns 0  carrier 0  collisions 0
```

inet 172.24.215.49
netmask 255.255.240.0

Note: Choose the IP addresses
based on what you see in your machine.

Change the HOST value (IP addr)
as shown below



```
Thonny - G:\Users\Mouli\RVU\Courses\NetSec\PMaterials\Labs\Lab1\code\Lab1_exp0_client_Win.py 4:77
File Edit View Run Tools Help
Lab1_exp0_client_Win.py *
1 import socket
2
3 # Server address (replace with the server's IP)
4 # Replace the below IP address based on what you see on WSL for ifconfig cmd
5 HOST = '172.24.215.49' # WSL: IP address assigned on Ubuntu
6
7 PORT = 65432 # Same port as the server
```

Exp 3a : Server (WSL) ☐ ☐ Client (Windows)

(file names: Lab3a_server_WSL.py & Lab3a_client_Win.py)

Run the server code of Exp0 on WSL and Client code of Exp0 on Windows

Server on WSL

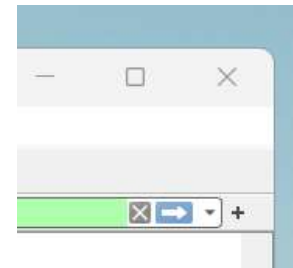
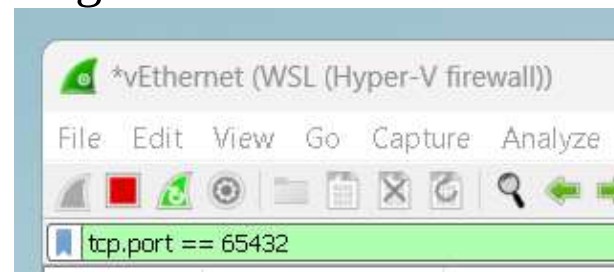
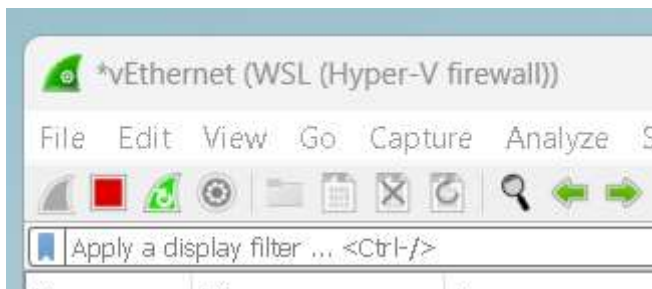
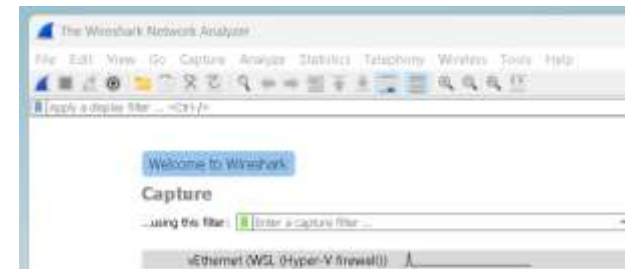
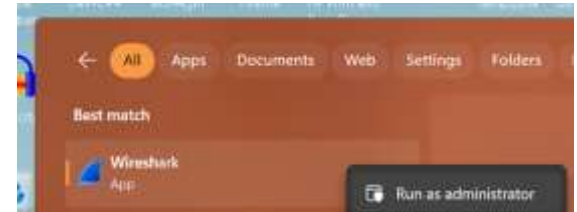
python3 Lab3a_server_WSL.py

Client on Windows

py Lab3a_client_Win.py

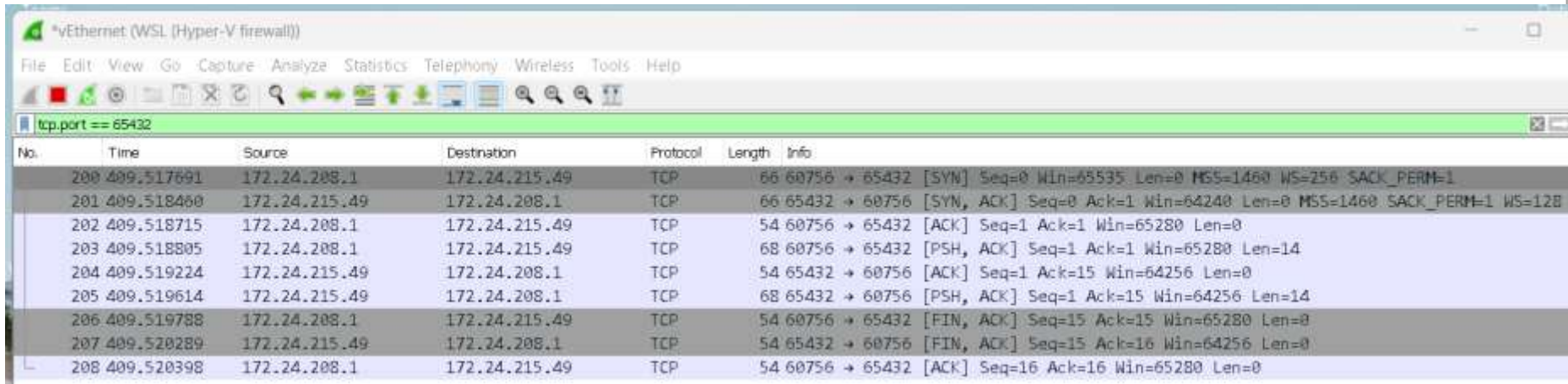
Analyze the TCP Segments: Wireshark (exchanged between the Server and Client)

- Run the **Wireshark in Administrator mode** and capture the packets on the virtual Interface between the WSL and Windows
- Choose: Capture **vEthernet Interface**
- Apply a **display filter**: Type the filter cmd
 - **tcp.port == 65432**
 - Click on the right arrow at the right side



- Run the Server code on WSL first and then the Client code on the Windows side.
- You can observe TCP segments exchanged between the Server and the Client on Wireshark.

Analyze Each TCP Segment Exchanged



The image shows a Wireshark packet capture window titled "vEthernet (WSL (Hyper-V firewall))". The filter bar at the top shows "tcp.port == 65432". The packet list table below shows 8 captured packets. The first three packets (200-202) represent the TCP three-way handshake. The next three packets (203-205) show data exchange (PSH, ACK). The final two packets (206-207) show the connection being closed (FIN, ACK). Packet 208 is an additional ACK packet.

No.	Time	Source	Destination	Protocol	Length	Info
200	409.517691	172.24.208.1	172.24.215.49	TCP	66	60756 → 65432 [SYN] Seq=0 Win=65535 Len=0 MSS=1460 WS=256 SACK_PERM=1
201	409.518460	172.24.215.49	172.24.208.1	TCP	66	65432 → 60756 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460 SACK_PERM=1 WS=128
202	409.518715	172.24.208.1	172.24.215.49	TCP	54	60756 → 65432 [ACK] Seq=1 Ack=1 Win=65280 Len=0
203	409.518805	172.24.208.1	172.24.215.49	TCP	68	60756 → 65432 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=14
204	409.519224	172.24.215.49	172.24.208.1	TCP	54	65432 → 60756 [ACK] Seq=1 Ack=15 Win=64256 Len=0
205	409.519614	172.24.215.49	172.24.208.1	TCP	68	65432 → 60756 [PSH, ACK] Seq=1 Ack=15 Win=64256 Len=14
206	409.519788	172.24.208.1	172.24.215.49	TCP	54	60756 → 65432 [FIN, ACK] Seq=15 Ack=15 Win=65280 Len=0
207	409.520289	172.24.215.49	172.24.208.1	TCP	54	65432 → 60756 [FIN, ACK] Seq=15 Ack=16 Win=64256 Len=0
208	409.520398	172.24.208.1	172.24.215.49	TCP	54	60756 → 65432 [ACK] Seq=16 Ack=16 Win=65280 Len=0

- The first three segments seen above are related to connection establishment between the Server waiting on the port # 65432
- The next three segments are exchanging data between the Server and the Client
- The last three segments are closing the connection between them, with the Client initiating the closure of connection



Experimentes to be Done in the Lab

Exp 3b: Server (Win) and Client (WSL)[Interchange Client & Server]

- Copy the given Exp0 files of both Server and the Client, as Exp1 files and change the Exp1 Client file according to your IP address observed on Windows
 - Exchanging files between the WSL and Windows seamless, use the method shown on Slide #5.
- Now, run the Exp1 Server on Windows first and run the Exp1 Client code from the WSL side and observe the segments exchanged between the WSL and Windows using the Wireshark.
- Observe the exchange of segments between the server running Windows and the client running on WSL on Wireshark, by setting up the display filter as “**tcp.port == 65432**”

Lab 3: Summary

- WSL Explained
- Server and Client Interactions
 - Using TCP Connection between WSL and Windows
 - Connection Establishment Explained
- Analyze the TCP Segments: Wireshark